

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 06/04

- | | |
|--------------------------|---------------------------------|
| ▪ Tipos de datos en C++. | ▪ Salidas de Datos Formateadas. |
| ▪ Variables. | ▪ Entrada de Datos. |
| ▪ Constantes. | ▪ Expresiones. |
| ▪ Expresiones. | ▪ Operadores relacionales. |
| ▪ Reglas de precedencia. | ▪ Operadores Lógicos. |
| ▪ Salidas. | ▪ Conversión de datos. |

2

Algoritmos y Estructuras de Datos – AEDD – Asistencia 06/04

3mmrx9

3

Tipos de datos en C++

Un programa trabaja con datos con unas determinadas características. No es lo mismo procesar números naturales, números reales o cadenas de caracteres.

En cada caso tratamos con datos de tipo diferente.

La elección de un determinado tipo u otro para una determinada entidad del programa proporciona información acerca de:

- El rango de posibles valores que puede tomar.
- El conjunto de operaciones y manipulaciones aplicables.
- El espacio de almacenamiento necesario para almacenar dichos valores.
- La interpretación del valor almacenado.

4

Tipos de datos en C++

En C++ los datos se clasifican en:

- **Simple:** valores indivisibles, es decir, no se dispone de operadores para acceder a parte de ellos: int, float, double, char, bool.

Por ej: si 3.1415 es un valor de tipo double, podemos usar el operador + para sumarlo con otro, pero no disponemos de operadores predefinidos para acceder a sus dígitos individualmente (aunque esto se pueda realizar mediante operaciones aritméticas).

- **Compuestos:** están formados como un agregado o composición de otros tipos, ya sean simples o compuestos: arreglos, cadenas, estructuras (struct)

5

Tipos de datos en C++ - Simples predefinidos

Tipo	Bytes	Bits	Min.Valor	Max.Valor
bool	1	8	false	true
char	1	8	-128	127
short	2	16	-32768	32767
int	4	32	-2147483648	2147483647
long	4	32	-2147483648	2147483647
long long	8	64	-9223372036854775808	9223372036854775807
unsigned char	1	8	0	255
unsigned short	2	16	0	65535
unsigned	4	32	0	4294967295
unsigned long	4	32	0	4294967295
unsigned long long	8	64	0	18446744073709551615
float (Precisión: 7 dígitos)	4	32	1.17549435e-38	3.40282347e+38
double (Precisión: 15 dígitos)	8	64	2.2250738585072014e-308	1.7976931348623157e+308
long double	12	96	3.36210314311209350626e-4932	1.18973149535723176502e+4932

6

Variables

- Son posiciones de la memoria donde se almacena un valor de un cierto tipo.
- En C++ toda variable debe declararse antes de ser usada, dándole un nombre (identificador) y asignándole un tipo.

Declaración de Variables:

<tipo-dato> <nombre-variable> = <valor-inicial/expresión>

- El valor inicial o expresión es opcional, pero debe ser válido (del tipo correcto). La declaración reserva espacio de memoria para almacenamiento y debe situarse al principio de un bloque.
- Si la declaración contiene el signo “=” se le proporciona un valor inicial a la variable Inicialización.
- Es posible acceder a cualquier variable declarada, pero si no se ha INICIALIZADO, referirá al valor (“basura”) que tiene almacenado el espacio de memoria.

Ejemplos de declaraciones de variables:

```
int suma = 0;
char genero;
int dni, dia, cantidad;
```

7

Constantes

C++ tiene 3 tipos de constantes:

- Constantes literales: se escriben directamente en el programa.

Pueden ser:

- Constantes booleanas: false, true
- Constantes enteras: 123, 1860L (long), 0773 (octal), 0AFF3 (hexadecimal)
- Constantes de coma flotante: 23.4, -3.1932, -1.76E12
- Constantes de caracteres: se encierran entre comillas simples: ‘a’, ‘Z’, ‘3’, ‘.’.
- Constantes de cadena: Las cadenas se encierran entre comillas dobles y se representan por una secuencia de caracteres, más un carácter nulo al final insertado automáticamente por el compilador: “es cadena”, “z”.

8

Constantes

Constantes definidas: son nombres (identificadores) asociados a valores literales constantes.

Se definen mediante: **#define <nombre> <valor>**

Ejemplo: **#define PI 3.141592** -- (no terminan con ;)

Constantes declaradas: son como una variable pero su valor no puede ser modificado.

Se definen mediante: **const <tipo-dato> <nombre-constante> = <valor-constante>;**

Ejemplos:

const int Meses = 12;

const int Semana = 7;

const float PI=3.141592; -- (terminan con ;)

9

Expresiones

- Un programa se basa en la realización de una serie de cálculos con los que se producen los resultados deseados.
- Dichos resultados se almacenan en variables y a su vez son utilizados para nuevos cálculos, hasta obtener el resultado final.
- Los cálculos se producen mediante la evaluación de expresiones en las que se mezclan operadores con operandos (constantes literales, constantes simbólicas o variables).
- En caso de que en una misma expresión se utilice más de un operador habrá que conocer la precedencia de cada operador (aplicando el de mayor precedencia), y en caso de que haya varios operadores de igual precedencia, habrá que conocer su asociatividad (si se aplican de izquierda a derecha o de derecha a izquierda).

10

Reglas de precedencia

Operador	Tipo de Operador	Asociatividad
[] -> .	Binarios	Izq. a Dch.
! ~ - *	Unarios	Dch. a Izq.
* / %	Binarios	Izq. a Dch.
+ -	Binarios	Izq. a Dch.
<< >>	Binarios	Izq. a Dch.
< <= > >=	Binarios	Izq. a Dch.
== !=	Binarios	Izq. a Dch.
&	Binario	Izq. a Dch.
^	Binario	Izq. a Dch.
	Binario	Izq. a Dch.
&&	Binario	Izq. a Dch.
	Binario	Izq. a Dch.
?:	Ternario	Dch. a Izq.

11

Significado de los operadores

	Operador	Significado
	[]	Selección de elemento en una tabla
	.	Selección de miembro de una tupla
	()	Paréntesis
	-	Cambio de Signo
	!	Negación
Aritméticos	*	Multiplicación
	/	División entera
	/	División real
	%	Módulo
	+	Suma
	-	Resta
Relacionales	<	Menor
	<=	Menor o igual
	>	Mayor
	>=	Mayor o igual
	==	Igual
	!=	Diferente
Lógicos	&	Conjunción lógica bit a bit
		Disyunción lógica bit a bit
	&&	Conjunción lógica
		Disyunción lógica

- Aritméticos
- Relacionales (comparaciones)
- Lógicos

12

Operadores C++

++	Incremento		Derecha a izquierda
--	Decremento		
-	Menos unario		
!	Negación lógica		
~	Complemento a uno		
(type)	Conversión de tipo (molde)		
sizeof	Tamaño del almacenamiento		
&	La dirección de		
*	Indirección		
*	Multiplicación	Aritméticos	Izquierda a derecha
/	División		
%	Módulo (residuo)		
+	Adición		Izquierda a derecha
-	Sustracción		
<<	Desplazamiento a la izquierda		Izquierda a derecha
>>	Desplazamiento a la derecha		
<	Menor que	Relacionales	Izquierda a derecha
<=	Menor que o igual a		
>	Mayor que		
>=	Mayor que o igual a		Izquierda a derecha
==	Igual a		
!=	Diferente a		
&	AND bit por bit		Izquierda a derecha
^	OR exclusivo bit por bit		Izquierda a derecha
	OR inclusivo bit por bit		Izquierda a derecha
&&	AND lógico	Lógicos	Izquierda a derecha
	OR lógico		
?:	Expresión condicional		Derecha a izquierda
=	Asignación		Derecha a izquierda
+= -= *=	Asignación		
/= %= &=	Asignación		
*= =	Asignación		
<<= >>=	Asignación		
,	Coma		Izquierda a derecha

13

Salida de datos

El estándar de salida (stdout) es la pantalla.

- En el archivo iostream están definidas macros, constantes, variables y funciones para la salida.
- La función cout deriva a pantalla los valores a mostrar:

```
cout << cadena;
```

```
cout << dato;
```

```
cout << endl;
```

- Los datos pueden ser: expresiones, variables, constantes.

14

Salida de datos Formateada

Para dar formato a la salida, se utiliza la biblioteca `iomanip`:

Salida de Datos

`#include <iomanip>`

▪ Manipuladores de formato:

fixed: Muestra un punto decimal y usa 6 dígitos después de éste.

setprecision (int p): Fija la precisión de salida, en p lugares.

Si se designa `fixed`, `p` especifica el número total de dígitos después del punto decimal; sino, `p` especifica el número total de dígitos desplegados (números enteros más dígitos fraccionarios), sin contar el punto decimal.

▪ `setw (int w):` Fija el ancho de salida, en w lugares.

▪ `setfill (int f):` Fija el carácter de relleno.

▪ `boolalpha:` Muestra valores booleanos como `true` o `false`, en lugar de como 0 y 1.

15

Salidas de datos formateadas

▪ En ocasiones puede interesar controlar la forma en la que se muestran los datos, especificando un determinado formato.

```
/* Salida de datos con diferentes formatos */
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    bool x = true;
    cout << boolalpha << x << endl; // escribe los booleanos como 'false' o 'true'

    cout << dec << 27 << endl; // escribe 27 (decimal)
    cout << hex << 27 << endl; // escribe 1b (hexadecimal)
    cout << oct << 27 << endl; // escribe 33 (octal)

    cout << setprecision(2) << 4.567 << endl; // escribe 4.6
    cout << dec << setw(5) << 234 << endl; // escribe " 234"
    cout << setfill('#') << setw(5) << 234; // escribe "##234"
    return 0;
}
```

```
true
27
1b
33
4.6
 234
##234
```

16

Entrada de Datos

- Los datos de entrada pueden tener diversas fuentes: teclado, archivos, etc.
- El estándar de entrada (stdin) es el teclado.
- En el archivo iostream están definidas funciones para la entrada.
- La función cin permite la entrada de datos:

`cin >> var1 >> var2;`



`cin >> var1;`

`cin >> var2;`

17

Expresiones

Asignación: Variable = expresión

Expresión: es un elemento de un programa que retorna un valor.

- Puede ser una constante, una variable o una fórmula que se ha evaluado, cuyo resultado se puede asignar a la variable de la izquierda.

Ejemplos: `a = 3*z + sum(x)` ; `a = 3`; `a = b`;

- La asignación destruye el valor anterior.
- La asignación puede tener notaciones abreviadas:
 - `a++`; // Incrementa en 1 el valor de a. Significa: `a = a+1`;
 - `a *= 3`; // Almacena en a el valor de `a*3`. Significa: `a = a*3`;
 - `a = b = c = 56`; // Almacena en cada variable el valor 56

18

Notaciones abreviadas de asignación

- En asignaciones donde se suma, resta, multiplica o divide, se puede usar una notación abreviada:

	Sentencia	Equivalencia
Operador prefijo de incremento/decremento	<code>++variable;</code>	<code>variable = variable + 1;</code>
	<code>--variable;</code>	<code>variable = variable - 1;</code>
Operador posfijo de incremento/decremento	<code>variable++;</code>	<code>variable = variable + 1;</code>
	<code>variable--;</code>	<code>variable = variable - 1;</code>
	<code>variable += expresion;</code>	<code>variable = variable + (expresion);</code>
	<code>variable -= expresion;</code>	<code>variable = variable - (expresion);</code>
	<code>variable *= expresion;</code>	<code>variable = variable * (expresion);</code>
	<code>variable /= expresion;</code>	<code>variable = variable / (expresion);</code>

19

Operadores pre y posfijos

Pensemos... ¿Cuál es la diferencia?

Ejemplo operador de prefijo: `valor = ++n;`

Ejemplo operador de posfijo: `valor = n++;`

¿Son lo mismo?

20

Operadores relacionales

Son: ==, !=, <, >, >=, <=

Forma de utilización:

Expresión1 **operador-relacional** Expresión2

Las expresiones y el operador deben ser compatibles. Sirven para comprobar una condición. Devuelven 0 (condición falsa) o 1 (condición verdadera).

Los caracteres se comparan utilizando el código ASCII. La comparación alfabética de cadenas debe hacerse con la función strcmp (cad1, cad2).

▪ Ejemplos de expresiones relacionales válidas:

b * b > 5 * c;

inicial != 'A';

medida == 12.6;

'a' < 'p'

21

Operadores Lógicos

También llamados operadores booleanos, son: ! (not) && (and) || (or)

- Las evaluaciones se corresponden con el álgebra de Boole y con sus prioridades.
- Se trabaja con evaluación en cortocircuito: se evalúa el operando de la izquierda y si éste determina en forma unívoca la expresión, el de la derecha no se evalúa.

Ej: x=0; if ((x>0) && (log (x) <8))

Ventajas:

- Ahorrar tiempo en la evaluación de condiciones complejas
- Evitar errores: (n!=0) && (x < 1.0/ n)

22

Operadores Lógicos

Los resultados de expresiones booleanas se pueden almacenar en variables mediante asignación:

```
int edad, MayorEdad;
```

```
MayorEdad = (edad > 18);
```

Como el operando de la izquierda es falso, la expresión completa es falsa → no se evalúa el operando de la derecha. Cuando `edad > 18` asigna 1 a `MayorEdad`, sino le asigna 0.

23

Operadores sizeof

Sizeof: proporciona el tamaño en bytes de un tipo de dato o variable.

- `sizeof (nombre_variable)` Ejemplo: `sizeof (a)`
- `sizeof (tipo_dato)` Ejemplo: `sizeof (int)`
- `sizeof (expresión)` Ejemplo: `sizeof (a + b)`

24

Ejemplo

```

/* Ingresar 3 nros enteros y mostrar su promedio */
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    int n1, n2, n3;

    /* Ingreso de los datos */
    cout << endl << "Ingrese el numero 1: ";
    cin >> n1;
    cout << endl << "Ingrese el numero 2: ";
    cin >> n2;
    cout << endl << "Ingrese el numero 3: ";
    cin >> n3;

    /* Impresion de resultados */
    cout << endl << endl << "El promedio de los valores ingresados es: "
    << setprecision(3) << setw(10) << ((float)(n1+n2+n3)/3) << endl;

    return 0;
}

```

```

Ingrese el numero 1: -3
Ingrese el numero 2: 4
Ingrese el numero 3: -2

El promedio de los valores ingresados es:    -0.333

```

25

Conversión de Datos

Frecuentemente se necesita convertir un valor de un tipo de dato a otro tipo de dato, sin cambiar el valor que representa.

Se hacen conversiones automáticas:

- Cuando se asigna un valor de un tipo a una variable de otro tipo.
- Cuando se combinan tipos mixtos en expresiones.
- Cuando se pasan argumentos a funciones. Promoción de tipos: char, int, unsigned int, long, unsigned long, float, double El tipo de dato que viene primero en esta lista se convierte al que viene segundo. Por ejemplo, si los tipos operandos son int y long, el operando int se convierte en long.
- Conversión Implícita (automática): cuando los tipos básicos están mezclados en asignaciones y expresiones.

Los datos de tipos más bajos se convierten a datos de tipos más altos. Ejemplo:

```
int i = 12;
```

```
double x = 4;
```

```
x = x + i; // antes de la suma i se convierte al tipo double
```

```
x = i / 5; // Primero se hace una división entera i / 5 == 2, luego el 2 se convierte a tipo double: 2.0 y se asigna a x
```

26

Conversión Explícita de datos

Cuando fuerza la conversión con un operador de conversión de tipos (cast):

Formato: (tipo-de-dato-al-cual-convertir) valor o variable

Ejemplo:

(float) i;

(int) 3.4;

precio = (int) 16.77 + (int) 23.99

Los operadores C++ respetan reglas de prioridad (precedencia) y de asociatividad

27

AEDD – Tareas semanales!!!! Calculadora

Asociarse al curso de Omega llamado:

Comision F-2026-Anual

Y resolver el problema habilitado:

Calculadora Perdida

28

AEDD – Otras tareas !!!!!

Capítulos 1, 2 y 3

Libro: “Fundamentos de Programación
con el Lenguaje de Programación C++” –
Benjumea-Roldan

Capítulos 1, 2 y 3

Libro: “C++ para ingeniería y ciencias” –
Gary J. Bronson, 2da Edición

29

Algoritmos y Estructuras de Datos – AEDD – Agenda 06/04

- | | |
|--------------------------|---------------------------------|
| ✓ Tipos de datos en C++. | ✓ Salidas de Datos Formateadas. |
| ✓ Variables. | ✓ Entrada de Datos. |
| ✓ Constantes. | ✓ Expresiones. |
| ✓ Expresiones. | ✓ Operadores relacionales. |
| ✓ Reglas de precedencia. | ✓ Operadores Lógicos. |
| ✓ Salidas. | ✓ Conversión de datos. |

¡Objetivo del día alcanzado!

30

Algoritmos y Estructuras de Datos – AEDD – Agenda 06/04 – Segunda parte

- Convenciones de nombres para variables y constantes
- Programación estructurada
- Estructuras de Control: Secuencia.
- Bloques.
- Declaraciones Globales y Locales.
- Estructura de control: Selección.
- Selección completa e incompleta.

31

Identificadores para variables y constantes

Deben cumplir dos tipos de convenciones:

- Reglas sintácticas.
 - Debe **comenzar** con una letra (a-z | A-Z) o un guión bajo (_).
 - Después del primer carácter, puede incluir dígitos (0-9), letras o guiones bajos.
 - Los identificadores son **case sensitive**, por lo que **Variable** es diferente a **variable**.
 - No se pueden usar las palabras reservadas del lenguaje: **int**, **return**, **char**, etc.
- Convenciones de estilo o buenas prácticas.
 - ¡Coloque nombres significativos/descriptivos!
 - CamelCase
 - Snake_Case
 - Mantenga la consistencia dentro de un proyecto.

32

Identificadores para variables y constantes

Respecto a las constantes:

- Respecto a los nombres, siguen las mismas reglas nombradas.
- Convenciones de estilo o buenas prácticas.
 - Suelen colocarse sus nombres iniciando en mayúsculas.
 - Una convención, muy utilizada, en las practicas propuestas por Google, es iniciar las constantes con la letra “k”.
 - Sea escribiendo en mayúsculas, o iniciando con “K”, siempre debemos ser consistentes en un proyecto.

33

Metodología de Programación Estructurada

Metodología que consiste en escribir programas con las siguientes reglas:

- El programa tiene un diseño modular
- Los módulos son diseñados de modo descendente
- Cada módulo se codifica utilizando las tres estructuras de control básicas:
 - Secuencia
 - Selección
 - Repetición

34

Metodología de Programación Estructurada

Teorema fundamental de la PE (Böhm y Jacopini, 1966):

“Todo programa propio puede ser escrito usando solamente las estructuras de control básicas: secuencia, selección y repetición”

Propio:

- Tiene un único punto de entrada y un único punto de salida;
- Existen caminos desde la E a la S que se pueden seguir y que pasan por todas las partes del programa;
- Todas las instrucciones son ejecutables y no hay bucles infinitos.

35

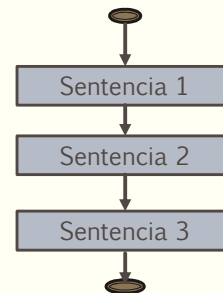
Estructuras de control

- Controlan el flujo de ejecución de las acciones de un programa
- Permiten combinar instrucciones o sentencias individuales en una simple unidad lógica con un único punto de entrada y un único punto de salida
- Se clasifican en tres tipos:
 - Secuencia (Composición secuencial)
 - Selección (Composición alternativa), y
 - Repetición (Composición iterativa).

36

Estructuras de control : SECUENCIA

- Conjunto de sentencias que se ejecutan una luego de la otra (flujo secuencial).



- En C++: conjunto de sentencias que finalizan con ';' y están encerradas entre llaves.

```

{
  int x, z, y;
  x = 1 + b;
  z = y * 28;
}
  
```

37

Estructuras de control : SECUENCIA

- Ejemplo 1: Se ingresan las notas de 3 parciales y se muestra la nota

```

int main() {
  int nota_parcial1, nota_parcial2, nota_parcial3;
  cout << "Ingrese la nota de los tres parciales de AEDD: " << endl;
  cin >> nota_parcial1 >> nota_parcial2 >> nota_parcial3;
  cout << endl << "La nota promedio de los parciales es: " << (nota_parcial1 + nota_parcial2 + nota_parcial3) / 3;
  return 0;
}
  
```

```

Ingrese la nota de los tres parciales de AEDD:
8
5
7
La nota promedio de los parciales es: 6
  
```

- Ejemplo 2: Se ingresan las coordenadas de un punto del plano y se debe informar la distancia al origen.

```

int main() {
  float punto_x, punto_y;
  cout << "Ingrese las coordenadas: " << endl;
  cin >> punto_x >> punto_y;
  cout << "La distancia al origen es: " << sqrt(punto_x * punto_x + punto_y * punto_y);
  return 0;
}
  
```

```

Ingrese las coordenadas:
3.9 4.5
La distancia al origen es: 5.95483
  
```

38

Bloque

- Un bloque es una unidad de ejecución mayor que una sentencia, y permite agrupar una secuencia de sentencias como una unidad.
- Para ello, formaremos un bloque delimitando entre dos llaves la secuencia de sentencias que agrupa.
- Es posible anidar bloques, es decir, se pueden definir bloques dentro de otros bloques.

```
int main()
{
    <sentencia_1> ;
    <sentencia_2> ;
    {
        <sentencia_3> ;
        <sentencia_4> ;
        . . .
    }
    <sentencia_n> ;
}
```

39

Declaraciones locales y globales

Se pueden declarar identificadores en diferentes bloques, pudiendo entrar en conflicto unos con otros, por lo que debemos conocer las reglas utilizadas en el lenguaje C++ para decidir a qué identificador nos referimos en cada caso.

Hay dos clases de declaraciones: **globales y locales**.

- **Entidades globales** son aquellas que han sido definidas fuera de cualquier bloque. Su ámbito de visibilidad comprende desde el punto en el que se definen hasta el final del archivo. Se crean al principio del programa y se destruyen al finalizar éste.
- **Entidades locales** son aquellas que se definen dentro de un bloque. Su ámbito de visibilidad comprende desde el punto en el que se definen hasta el final de dicho bloque. Se crean en el punto donde se realiza la definición, y se destruyen al finalizar el bloque.

Normalmente serán variables locales, aunque también es posible declarar constantes localmente.

40

Declaraciones Globales y Locales

```
#include <iostream>
using namespace std;
const double EUR_PTS = 166.386;           //Declaración de constante GLOBAL

int main() {
    cout << "Introduce la cantidad (en euros): ";
    double euros;                          //Declaración de variable LOCAL
    cin >> euros;
    double pesetas = euros * EUR_PTS;      //Declaración de variable LOCAL
    cout << euros << " Euros equivalen a " << pesetas << " Pts" << endl;
    return 0;
}
```

```
Introduce la cantidad (en euros): 147
147 Euros equivalen a 24458.7 Pts
```

```
<< El programa ha finalizado: codigo de salida: 0 >>
<< Presione enter para cerrar esta ventana >>
```

41

Declaraciones Globales y Locales

- En caso de tener declaraciones de diferentes entidades con el mismo identificador en diferentes niveles de anidamiento, la entidad visible será aquella que se encuentre declarada en el bloque de nivel de anidamiento más interno.
- La declaración más interna oculta a aquella más externa.

```
int main() {
    int x = 3;
    int z = x * 2; // x es vble de tipo int con valor 3
    {
        double x = 5.0;
        double n = x * 2; // x es vble de tipo double con valor 5.0
    }
    int y = x + 4; // x es vble de tipo int con valor 3
    return 0;
}
```

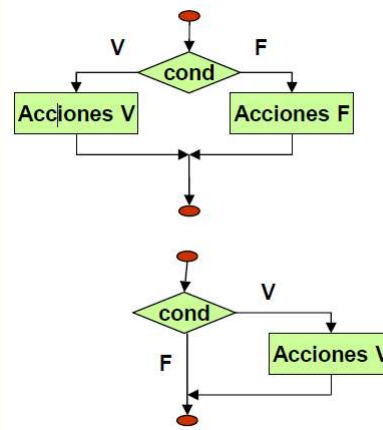
- No es una buena práctica de programación ocultar identificadores al redefinirlos en niveles de anidamiento más internos, ya que conduce a programas difíciles de leer y propensos a errores.

¡Evitar esta práctica!

42

Estructura de control: Selección

- **Selección completa:** Se evalúa una expresión booleana y según sea verdadera o falsa, se ejecuta una acción u otra; por cualquier caso la ejecución continúa con la sentencia siguiente al if.



- **Selección incompleta:** Sólo se desea realizar una acción en caso de que la expresión booleana sea verdadera y no hay acción por el falso.

Las acciones pueden ser una sentencia que termina en “;” o un grupo de sentencias encerradas entre llaves.

43

Estructura de control: Selección

- En C++:

SELECCIÓN
COMPLETA

```
if (expresión)
    acción_verdadero;
else
    acción_falso;
```

SELECCIÓN
INCOMPLETA

```
if (expresión)
    acción_verdadero;
```

44

Estructura de control: Selección

Practica para realizar: Selección Incompleta: Modifique el ejemplo 1, y solo muestre el promedio de las notas del parcial, si es mayor o igual a 7, indicando que se ha promocionado la materia

Otra práctica: Selección Completa: Modifique el código anterior, indicando el promedio si la nota es mayor o igual a 7, informando que se ha promocionado la materia, y en caso contrario, mostrar un mensaje que indique que no se ha promocionado la materia.

45

Estructura de control: Selección Completa

Otra práctica: Selección Completa: Modifique el ejemplo 2, validar que el punto no sea el origen y mostrar la distancia al origen.

```
Ingrese las coordenadas:
1
0
La distancia de 1,0 al origen es: 1
```

```
Ingrese las coordenadas:
0
0
Incorrecto, ha ingresado el origen
```

46

Algoritmos y Estructuras de Datos – AEDD – Agenda 06/04 – Segunda parte

- ✓ Convenciones de nombres para variables y constantes
- ✓ Programación estructurada
- ✓ Estructuras de Control: Secuencia.
- ✓ Bloques.
- ✓ Declaraciones Globales y Locales.
- ✓ Estructura de control: Selección.
- ✓ Selección completa e incompleta.

¡Objetivo del día SUPERADO!

47

AEDD – Otras tareas !!!!!

Capítulo 4

Libro: “Fundamentos de Programación con el Lenguaje de Programación C++” – Benjumea-Roldan

Capítulo 4

Libro: “C++ para ingeniería y ciencias”, 2da Edición. Gary J. Bronson

Capítulo 4

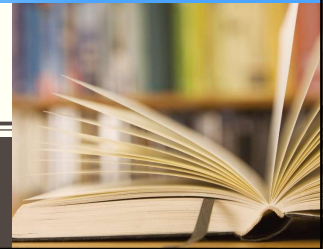
Libro: “Cómo programar en C++” – Harvey M. Deitel y Paul J. Deitel

48

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026– Cursado Anual

¿DUDAS?



49

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Buena semana!



50